

DOORS

UI-to-Master API Encryption and Decryption Process

Interactive portal session protection for encrypted browser response payloads

Scope.

Defines the current browser-to-Master API contract. Interactive authentication is carried by the Secure, HttpOnly DOORS_SESSION cookie. The browser does not read or attach the session token; the UI receives only a scoped sessionCryptoKey from /api/v1/auth/me for response decryption.

1. Authentication and request boundary

- Browser requests use same-origin HTTPS with credentials enabled; the Secure, HttpOnly session cookie is attached by the browser.
- The shared Axios client removes Authorization and X-API-KEY headers from interactive portal traffic.
- State-changing requests use the XSRF token obtained from /api/v1/auth/csrf.
- Plane routing and role checks run before business authorization; admin, external and machine API surfaces remain separated.

2. Session cryptographic context

After authenticated session bootstrap, /api/v1/auth/me returns the canonical user profile plus sessionCryptoKey. The value corresponds to the server-side session response-encryption context but does not expose the HttpOnly session token to JavaScript.

Component	Receives	Must not receive/store
Browser cookie jar	Secure, HttpOnly DOORS_SESSION	Application logs, URL/query string, localStorage
Vue user state	sessionCryptoKey for the active session	Raw session/JWT token or server private keys
Master API	Authenticated session context	Unminimized entities or secrets

3. Master API browser-response protection

1. **Authorize.** Validate the portal session, plane, role and resource-specific permission.
2. **Minimize.** Project the result to approved DTO/contract fields; remove password hashes, session IDs, private keys and internal configuration.
3. **Serialize.** Serialize the safe payload once to UTF-8 JSON.
4. **Derive.** Build the current 16-byte session response key on the server from the authenticated session token context.
5. **Encrypt.** Encrypt the serialized payload using AES/ECB/PKCS5Padding and Base64-encode the ciphertext.
6. **Envelope.** Return isEncryptedPayload=true and secureData inside the normal API response envelope, with only non-sensitive metadata outside ciphertext.

```
{ "status": "SUCCESS", "message": "...", "payload": { "isEncryptedPayload": true, "secureData": "<Base64 ciphertext>", "rowCount": 1 } }
```

4. UI response decryption and rendering

1. **Validate.** Locate payload/data/body envelope and require isEncryptedPayload=true plus non-empty secureData.
2. **Read context.** Use currentUser.sessionCryptoKey loaded by authenticated /auth/me bootstrap; never attempt to read the HttpOnly cookie.
3. **Decode.** Base64-decode secureData into ciphertext bytes.
4. **Decrypt.** Use AES/ECB with PKCS7 padding (browser-library equivalent of PKCS5 for AES) and decode UTF-8.
5. **Parse.** Parse JSON, validate the expected list/object shape and map only contract fields into component state.
6. **Dispose.** Keep plaintext only as long as needed for the active UI; never place it in logs, URLs, analytics or persistent storage.

Reference sequence: authenticated cookie request -> authorization/minimization -> AES encrypted payload -> sessionCryptoKey from user state -> browser decrypt -> validate -> render.

5. Failure handling

Failure	Required behavior
Session expired or missing	Return 401, clear user state and redirect to login through the central response interceptor
Wrong portal/role	Return controlled 403; never disclose protected payload
Missing sessionCryptoKey	Stop decryption and report that the session cryptographic context is unavailable
Malformed envelope/Base64/padding	Reject response; do not accept plaintext fallback
Invalid JSON/shape	Do not render; show a generic operation error and retain only correlation-safe diagnostics

6. Security limitations and migration guidance

- This application-layer encryption supplements TLS; it does not protect plaintext after authorized JavaScript decrypts it for rendering.
- A compromised browser, malicious extension or injected script in the same execution context may observe decrypted data and sessionCryptoKey.
- AES-ECB provides confidentiality compatibility but no nonce or authentication tag. New contract versions should migrate to AES-GCM with a unique nonce and explicit version metadata.
- Do not use the legacy fixed-key fallback in new code. Primary portal components must require currentUser.sessionCryptoKey.
- Content Security Policy, dependency integrity, output encoding, secure cookies, short session lifetime and logout invalidation remain essential.

7. Acceptance checks

- Interactive requests contain no Authorization or X-API-KEY header and authenticate successfully with the Secure, HttpOnly cookie.
- /auth/me supplies sessionCryptoKey only after authentication and the UI clears it on logout/session failure.
- Protected endpoints never return domain arrays/objects in plaintext when encryption is required.
- UI decryption succeeds with the active session context and fails with a stale or missing context.

- Password hashes, session IDs, private keys and database credentials are absent before encryption.
- No plaintext or session cryptographic material appears in logs, URLs, browser storage or telemetry.

8. Implementation references

Responsibility	Current implementation
Cookie-only portal client	src/api/axios-instance.ts
Session user/bootstrap context	src/utls/userState.ts and AuthController /auth/me
Shared response decryptor	src/utls/hybridResponseCrypto.ts
Protected UI flows	QuerySubmission, PendingQueries, ReportViewer and governance controllers

End of controlled document.